Recitation #3

Part A – Big Brother

We have a set of individuals we are tracking via GPS. Each measurement record consists of a 4 byte id for the individual, an 8 byte encoding of the GPS position, 8 byte encoding of error/device state and 8 bytes for a time stamp. Positions are collected from each device every minute (note these are not synchronized so one device might give the measurement at 773.111 milliseconds into a second and another at 212.043 milliseconds in). Assume that disk blocks are 8 KB and with 64-bit addresses. Further assume that B+tree vertices fit into exactly one disk block and have 64-bit addresses (Thus all pointers are 8 bytes). We cluster on time stamps and build a sparse index on time stamps. You may assume that timestamps form a primary key (no two devices report on exactly the same time stamp). Assume we have been tracking 10 000 people for 100 days.

1. For the measurement data, what is the blocking factor for a B+ tree leaf node with forward and backward pointer to next/previous blocks.
2. What is the order of an internal B+tree vertex?
3. How many measurement records are collected and how many blocks required assuming full data record blocks?
4. What is the maximum possible height of a primary B+tree built over time stamps assuming full data record blocks? (The leaf vertices are maximally packed while the interior vertices are as empty as possible)
5. For 4 above, what is the minimum height (interior nodes maximally packed as well as leaf nodes)?
6. We build a secondary B+tree index on (id, time stamp). Assume leaf nodes are 1 block buckets that contain a sequence of timestamps and include pointers to the next and previous buckets. Note that the records associated with these timestamps could be accessed via the primary index. Assume these buckets are completely full. What is the maximum height of such an secondary index?
7. In words what would the Index of 6 let you query for/do?
8. Could this be implemented on a single server with a 100 Mb/s network connection? (assume that packets have minimal overhead and they are distributed randomly over a one second interval, there are no confirmation receipts, each device has a separate IP)

Part B – My God, it's Full of Stars!

Study and then load the schema here into a new PostgreSQL database: schema

Load this data into that database: data

Solve the following in SQL and analyse via `EXPLAIN ANALYSE`. Can you build indexes, re-cluster, etc. to improve performance?

9. Stars with spectral type 'M' in constellation Pisces.
10. The 100 stars closest to the sun.

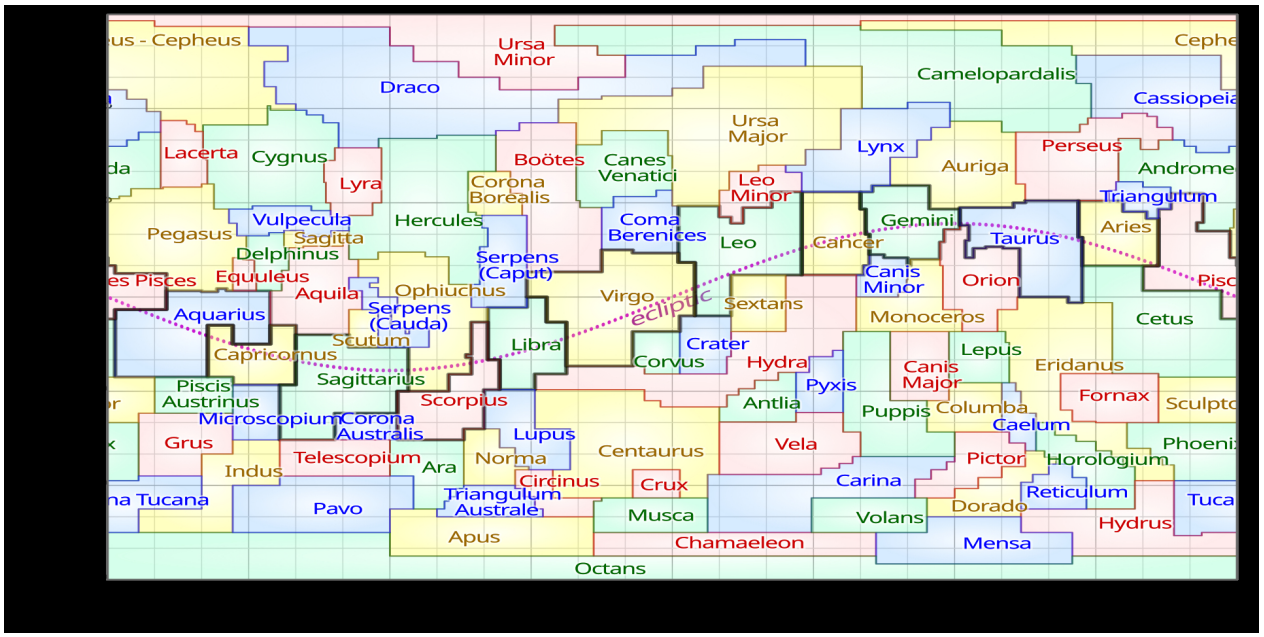


Figure 1: Constellation boundaries with the ecliptic.

11. The 100 stars closest to the star with proper name Antares (do not plug in the coordinates of this star in SQL. The literal 'Antares' must be in your SQL)
12. The 100 closest exoplanets to the sun. Measure this based on the distance of the exoplanets star.
13. How many exoplanets per constellation?
14. There is star that is not placed in its correct constellation. Which star is that?
15. Give the stars within a distance of 1.0 pc's of a star with spectral type 'M'
16. Give the stars not within a distance of 50 pc of a star with spectral type 'M'
17. This will take some time, but make 1 million new galaxies via:

```
select make_galaxies(1000000);
```

18. Give the dso's in virgo with a redshift greater than .5
19. Give the exoplanets that seem most likely to support human life.
20. Give an example of the simplest query you can think of that will not return within hours if not days...